

SSDTrain: An Activation Offloading Framework to SSDs for Faster Large Language Model Training

Kun Wu^{*†‡}, Jeongmin Brian Park^{*†‡}, Xiaofan Zhang^{*‡},

Mert Hidayetoğlu[§], Vikram Sharma Mailthody[†], Sitao Huang[¶], Steve Lumetta^{||}, Wen-mei Hwu^{†‡}
[†]NVIDIA [‡]Google [§]Stanford University [¶]University of California, Irvine ^{||}University of Illinois Urbana-Champaign

^{*}These authors made equal contributions

Abstract—The growth rate of the GPU memory capacity has not been able to keep up with that of the size of large language models (LLMs), hindering the model training process. In particular, activations—the intermediate tensors produced during forward propagation and reused in backward propagation—dominate the GPU memory use. This leads to high training overheads such as expensive weight update costs due to the small micro-batch size. To address this challenge, we propose SSDTrain, an adaptive activation offloading framework to high-capacity NVMe SSDs. SSDTrain reduces GPU memory usage without impacting performance by fully overlapping data transfers with computation. SSDTrain is compatible with popular deep learning frameworks like PyTorch, Megatron, and DeepSpeed, and it employs techniques such as tensor deduplication and forwarding to further enhance efficiency. We extensively experimented with popular LLMs like GPT, BERT, and T5. Results demonstrate that SSDTrain reduces 47% of the activation peak memory usage. At the same time, SSDTrain perfectly overlaps the I/O with the computation and incurs negligible overhead. Compared with keeping activations in GPU memory and layerwise full recomputation, SSDTrain achieves the best memory savings with negligible throughput loss. We further analyze how the reduced activation memory use may be leveraged to increase throughput by increasing micro-batch size and reducing pipeline parallelism bubbles.

I. INTRODUCTION

LLMs now drive a wide range of applications, including chatbots [1], search [2], content generation [3], reasoning [4], etc. These models, when sufficiently large in size, demonstrate emergent abilities [5] and thus the capability of handling complicated tasks. Such a phenomenon drives model designers to continue to scale up the size of LLMs, carrying more parameters. The already formidably high training costs continue to grow: training GPT-4, for example, cost US\$100 million, a $21\times$ increase over training GPT-3 [6].

GPU memory capacity has become a bottleneck for the continued growth of LLMs. As Fig. 1 shows, the increase of GPU memory capacity is around 60% slower than the LLM size scaling speed and the GPU FP16 throughput improvement. About 80% of the GPU memory used to train recent LLMs consists of activations [7], [8], the intermediate tensors produced by forward propagation and reused in backward propagation. Furthermore, the memory needed for activations is growing more rapidly than any other memory use, making GPU memory a more serious constraint for future LLM training (see Sec. II-B for details).

Common mitigations are to reduce batch size or use gradient accumulation. With gradient accumulation, a batch is divided into micro-batches that are processed separately between gradient updates. Although gradient accumulation has been adopted by many LLMs [9]–[11], the GPU computation stack is not designed for small inputs, and both mitigations lead to device under-utilization [12], [13] and suboptimal math library performance [14]. Intuitively, a smaller batch size might reduce total training computation through

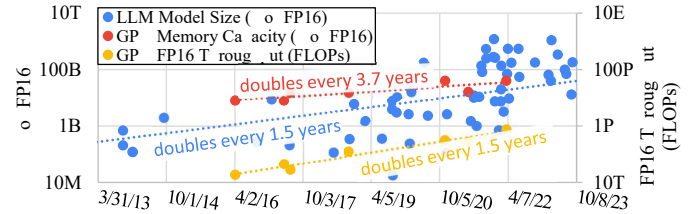


Fig. 1. The growth of FP16 throughput of GPUs for deep learning training is aligned with the model size of LLMs, but GPU memory capacity falls behind [18]. Horizontal axis shows release date. Points represent both Nvidia 100-level GPUs since K100 and Google TPUs.

faster convergence. However, LLM trainers have identified a critical batch size to each model, below which convergence speed increases negligibly or even decreases [15], [16]. Notably, critical batch size grows during training, as training loss is reduced.

Another common approach to reducing GPU memory use is activation checkpointing. With this, only some activations are kept in GPU memory, while others are flushed and then recomputed during backward propagation. For an L -layer model, activation checkpointing reduces memory requirements from $O(L)$ to $O(\sqrt{L})$ [17]. However, as Sec. II-B shows, even this alone is insufficient to eliminate the bottleneck posed by GPU memory limits for future LLMs.

This work proposes SSDTrain, a software framework that offloads activations to NVMe SSDs and reloads activations just before they are needed in backward propagation. SSDTrain is able to overlap activation transfers fully with computation, thereby reducing activation memory usage without incurring significant performance overhead. SSDs are a more attractive target than main (CPU) memory for several reasons. First, clusters and cloud instances [19]–[21] are typically limited in host memory capacity (100–250 GB/GPU), while SSDs offer much higher capacity. The limited host memory is also consumed by input, metadata, etc., which further reduces the amount of memory available for activation offloading. Second, host memory bandwidth is shared across training management tasks and offloaded computation [22]–[24] running on the host CPU and can be quite limited and even unpredictable [25] for saving and restoring activations. In contrast, the SSD bandwidth can be dedicated to the activation offloading during training. Third, SSDs are more elastic, both by adding more SSDs and even PCIe switches if necessary—as well as through the use of optional remote high-throughput storage [26], [27]. Such elasticity allows data centers to keep up with the fast-growing size of activations. In contrast, the memory capacity of GPU cloud instances and cluster nodes is much more difficult to extend.

This work makes the following main contributions.

1. We design and implement the SSDTrain framework to offload LLM activations to NVMe SSDs. We demonstrate the viability of SSDTrain on large-scale systems by modeling the performance,

The code repository is available at <https://github.com/K-Wu/FlashTrain>. Corresponding author: kunw@nvidia.com.

SSD lifespan and the required per-GPU PCIe bandwidth.

2. With all code in Python except for a tiny CUDA API hooking library, SSDTrain works with the latest PyTorch, and distributed frameworks. We developed and tested SSDTrain with Megatron-DeepSpeed [28] on a 2-GPU node with 7× Intel Optane SSDs.
3. Evaluation shows SSDTrain matches the original system’s training time while reducing the activations peak memory use by up to 47%. This proves that SSDTrain overlaps the data transfer fully with computation. Compared with keeping activations and layer-wise full recomputation, SSDTrain obtains the best performance and the least memory peak. We further analyze how the reduced activation memory use may increase throughput by increasing micro-batch size and reducing pipeline parallelism bubbles.

II. BACKGROUND

A. Transformer-Based LLM

Most LLM architectures, including GPT [29], are transformer-based [30]. These models consist mainly of transformer layers. Each transformer layer is primarily made up of an attention block and a multi-layer perceptron (MLP) block. GPT is a decoder-only model because it only involves transformer decoder layers. An encoder layer has the same structure as the decoder layer except that the latter imposes causality on the attention mask. Transformer models are classified as (1) encoder-only, e.g., BERT [31], (2) decoder-only, and (3) encoder-decoder, e.g., T5 [32]. In encoder-decoder models, decoder layers take in both outputs from the encoders and another text and apply two attention blocks—the self-attention block is applied to the new text, and the cross-attention block is applied among the tokens in the sequence from the encoder and tokens in the new text.

Parallelizing LLM training involves partitioning and/or replicating the model and the data into different GPUs [33]. Pipeline parallelism (PP), data parallelism (DP), and tensor parallelism (TP) are the three widely-adopted levels of parallelism available to all LLM models. PP divides the model and places chunks of layers on different GPUs. In a step, when the GPUs finish their layers, the output is passed to the GPUs owning next layers. DP replicates the models in different groups of GPUs and assigns separate micro-batches to each group. TP shards a weight tensor and puts shards onto different GPUs. Each GPU performs a portion of the computation using its shard for the corresponding operator. Zero Redundancy Optimizer (ZeRO) [34] further reduces memory use with DP by sharding the optimizer states, and/or optionally the gradients and parameters across these GPUs.

B. GPU Memory Capacity and Model Throughput

As Fig. 7 of Sec. IV will show, the GPU memory capacity is limiting the model throughput. By offloading the activations to SSDs, SSDTrain can alleviate this limitation and improve the per-GPU model throughput. An important question is if the GPU memory capacity will continue to be the limiting factor of per-GPU model throughput according to the trend of LLM scaling. This section shows that the historical trend will make GPU memory capacity an even more important limiting factor of the per-GPU model throughput.

Neural scaling laws [15], [16], [35] guide LLM scaling as computing power increases. We follow these laws in our reasoning. The whole-system GPU compute throughput $C \propto ND_{batch}$, where N is the number of parameters and D_{batch} is the number of tokens in a batch [36]. Chinchilla Scaling [35] concludes that the optimal model design follows $N \propto C^{0.5}$, which implies $D_{batch} \propto C^{0.5}$ to saturate the GPU throughput. Whole-system GPU memory use consists of two parts: activations, which require $S_{activations} \propto \frac{N}{h}D_{batch}$,

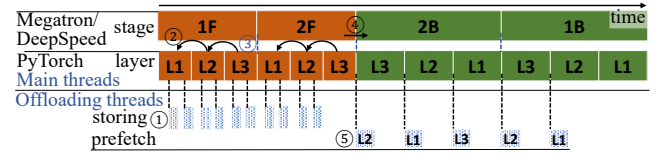


Fig. 2. SSDTrain timeline of a step of a 2-microbatch 3-layer (L) model.

where h is the hidden dimension in the layers and is a slow growing function of N , e.g., $h \propto N^{1/3}$, and all other memory use, $S_{others} \propto N$, including parameters, gradients, and optimizer states. Comparing the factors, we can deduce that (1) $S_{activations}$ grows faster than S_{others} , and (2) whole-system memory use, which is dominated by the activations, grows slightly slower than the compute throughput C (approximated $C^{5/6}$). However, Fig. 1 shows that the GPU memory capacity historically grows (red dotted line) at 41% the growth rate of the compute throughput (yellow dotted line). Therefore, **GPU memory capacity will become increasingly inadequate for saturating the compute throughput, and memory for activations will continue to dominate the GPU memory usage.**

What about activation checkpointing? Revisiting the prior equation, $S_{activations} \propto \frac{N}{h}D_{batch} \propto LhD_{batch}$ where L is the number of layers. Checkpointing makes the new activations memory use $S'_{activations} \propto \sqrt{LhD_{batch}}$. Since L and h grow when N increases and $D_{batch} \propto C^{0.5}$, $S'_{activations}$ still grows faster than S_{others} .

C. SSD Endurance

Trends in price, latency, and bandwidth have led to the widespread adoption and integration of SSDs into cloud instances and clusters [19]–[21]. Flash’s random write latency is reduced to tens of microseconds [37], and NVMe SSD data rates are now a few GB/s.

SSD endurance remains a concern: how long will SSDs last in the write-intensive activation offloading? SSD endurance is determined by the type and number of cells, write amplification factor (WAF), and over-provisioning. SSD cells can be purposed to store one bit or multiple levels. Generally, the more bits a cell stores, the shorter its lifetime in program-erase (PE) cycles. WAF is the ratio of media write amount to host write amount—SSD writes pages at a time but erases blocks of pages, a coarser granularity. Erasing a partially empty block requires that remaining valid pages be relocated, causing write amplification. In turn, vendors adopt over-provisioning to reserve some blocks for wear leveling, evening out the writes across blocks.

Notably, SSD endurance rating uses the JESD testing method [38] which performs random writes after tough preconditioning. In our scenario, the writes are large and sequential as each tensor being offloaded is easily hundreds of MBs in size. Such writes are more endurance-friendly compared with the writes used to determine the JESD rating. For example, 3-DWPD SSDs generally allow about 2.5× as many sequential writes than expected from the JESD rating [39]–[41]. Vendor guidelines [42]–[44] and empirical data [45] corroborate this difference. Sec. III-D uses modeling to demonstrate why mainstream data center SSDs are viable options to support SSDTrain deployment in a large-scale LLM training system.

D. SSD Offloading Systems for LLM

GPUDirect Storage (GDS) enables a direct data path between GPU and NVMe SSDs [48]–[50], removing the need for a CPU bounce buffer to enhance bandwidth and reduce both latency and CPU load.

To mitigate the training overhead caused by the GPU memory capacity limit, SSDTrain has three key differences with related work [50], [51]: SSDTrain offloads (a) activations to (b) the SSDs

TABLE I

COMPARING SSDTRAIN TO OTHER LLM SYSTEMS WITH ACTIVATION OFFLOADING FEATURES [46]–[48]. WITHOUT BACKWARD PROPAGATION, INFERENCE SYSTEMS MAY DISCARD INTERMEDIATE TENSORS ONCE A LAYER IS DONE. WE GENERALIZE “ACTIVATION” TO REFER TO KEY-VALUE (KV) CACHE AS WELL BECAUSE IT IS REUSED ACROSS STEPS.

	Flexgen	LLM in a Flash	ZeRO-Infinity	SSDTrain
Training				
Activation to main memory	✓	✓	Checkpoints only	✓
Offloading to SSD	✓			✓
Direct GPU-SSD data path				✓
Async data transfer				✓
Interoperability				✓

(c) with negligible performance overhead. To the best of our knowledge, SSDTrain is the first work that leverages SSD to offload activations for LLM training. Table I illustrates some other SSDTrain’s features:

Direct and async GPU-SSD data transfer. As Sec. I shows, transfer via CPU impacts efficiency. Besides, existing systems either block the training when loading offloaded data, or synchronize at each layer. Thus, the I/O latency is in the critical path. SSDTrain hides the I/O latency by overlapping I/O with computation.

Interoperability. Since LLM training requires a synergy of Python packages and the ecosystem is rapidly evolving, it is vital for offloading to have good interoperability with other components. SSDTrain logic is local to processes and can work with distributed frameworks, e.g., Megatron. In contrast, DeepSpeed’s offloading features, e.g., ZeRO-Infinity, are available only in certain ZeRO stages. ZeRO stage determines what is sharded. For example, stage-3 ZeRO in Fig. 5 shards optimizer states, gradients, and weights across the GPUs.

III. DESIGN AND IMPLEMENTATION

A. Overview of the SSDTrain Framework

SSDTrain implements a *tensor cache* to manage the offloading and reloading of tensors, facilitating the release of memory as well as the prefetch of tensors back to memory before they are needed for backward propagation. Fig. 2 exemplifies how SSDTrain works. SSDTrain launches its own threads to store tensors (①) and to reload them (⑤). In forward propagation (F), offloading of an activation starts once the operator producing it finishes (①). When activations are reused in backward propagation (B), prefetching (⑤) occurs in the reverse order of layers as recorded during forward propagation (②). If the last layer begins backward propagation immediately after its forward propagation (L3 in micro-batch 2 in the example), its activations are kept (④). SSDTrain keeps individual records for each micro-batch. Upon micro-batch changes (②), SSDTrain switches its own record to the one corresponding to the new micro-batch.

Fig. 3 shows the SSDTrain workflow. SSDTrain retrieves the amount of computation and activation size of the model from the model instance, GPU throughput, and SSD bandwidth. Then, SSDTrain sets the activation offload amount accordingly. The tensor cache manages the activations and performs tensor offloading and loading. To achieve this, it uses PyTorch hooks to alter PyTorch execution. Sec. III-B details the design and implementation of the tensor cache. SSDTrain has the SSD offloader that targets NVMe SSDs within the same node and the CPU offloader that targets host memory. Each offloader encapsulates the logic to transfer CUDA

tensors to and from a target. The SSD offloader leverages the GDS python binding, kvikio [52]. Using the LD_PRELOAD interposition mechanism, the CUDA malloc hook library alters CUDA memory allocation and free API calls so that the memory is properly registered and deregistered for best GDS performance. This allows us to keep the PyTorch memory allocator for easy comparison with the baseline, without replicating its implementation in a PyTorch pluggable memory allocator or modifying the PyTorch C++ code. The CPU offloader is for future work on clusters with massive remote SSD storage. It is backed by an allocator with pre-allocated host-pinned memory. The pool size is determined by profiling the first training step. Hints are added to Megatron’s and DeepSpeed’s schedulers, e.g., ③ and ④ in Fig. 2. E.g., for DeepSpeed’s scheduler, hints are added before and after the execution of each command, e.g., computing the micro-batch i , communication so that the tensor cache gets notified about the upcoming stage and the completion of an action. Accordingly, the tensor cache can prefetch data, or wait for I/O to complete.

To use SSDTrain, a few lines are to be added in the existing script: They register the PyTorch hooks, bookkeep the weights to not offload them, and monkey-patch [53] the schedulers.

SSDTrain extends naturally to distributed settings such as use with ZeRO, because frameworks such as DeepSpeed and Megatron divide the workload into processes built on top of PyTorch’s built-in tensor functionality. By working below PyTorch and keeping each process’s activities local, SSDTrain applies directly to distributed launches.

B. Hook-Based Implementation of Tensor Cache

To benefit from tensor offloading, the GPU memory that the offloaded tensors own must be released when the tensors are not in use. However, PyTorch by default stores a reference to all the activations on the computational graph, disallowing the GPU memory to be reclaimed. The tensor cache alters the PyTorch execution so that the identifiers of the activations are registered on the computation graph; upon PyTorch’s reusing the activation tensor, the tensor cache uses the identifier from the computational graph as the key to return the requested tensor. In the forward propagation, when the tensor finishes offloading, the tensor cache no longer holds a reference to it, allowing its memory to be reclaimed by Python garbage collection once the control flow gets out of the function scope where the tensor object is used. In the backward propagation, the tensor cache holds a reference to the tensor by loading it from the SSD before its use; when all the module scopes the tensor is referred to have been finished, the reference is no longer held, allowing its memory to be reclaimed. In short, the tensor cache is the in-memory structure that manages the references to all activations and tracks activations’ states, including if they are being offloaded, the path in the file system, etc.

Tensor cache uses PyTorch hooks to alter its execution behavior. The forward hook pair works in the forward propagation: The start of a module triggers the forward pre hook, and the finish of a module triggers the forward hook. Tensor cache maintains the current scope stack using the forward hook pair: Upon entrance to a module, the module is pushed to the stack; Upon module exit, it is popped out.

Backward hook pair is similar. When entering a module, the tensor cache prefetches activations in upcoming modules. Sec. III-C2 details prefetching. When exiting a module, the tensor cache removes it from the scope lists of all activations. Activations no longer in use are removed, whose memory will be released by garbage collection.

When a tensor is to be registered onto the computation graph, the pack hook is called to produce a value to be registered instead. When the tensor is reused, the unpack hook is called to take in the object on the computation graph and return the original tensor.

Fig. 4 illustrates tensor cache’s activity when pack or unpack hook is triggered. When the multiply operator $x \cdot w$ finishes (①), the pack hook is called (②) on the input x and weights w . Tensor cache has a record of weights, and accordingly returns w to let it be registered on the graph as is. The tensor will also be returned as is if the tensor is on CPU or it is too small (Line 2 in Alg. 1). As line 6 in Alg. 1 shows, the tensor cache does not offload tensors but only keeps a record when the module is to be kept in the memory or in backward propagation. The first condition holds true when the amount of activation in this step before the current tensor reaches the size set in Fig. 3. The second condition is true when an activation-checkpointing-enabled function does recomputation in the backward propagation to reproduce the activations. For tensor x in Fig. 4, the tensor cache stores it to the SSDs (③), updates the amount of activations offloaded in this step, and returns a tensor identifier. When the unpack hook is triggered (④), in the backward propagation (⑤), the tensor cache either waits until the prefetch finishes (⑥), and eventually returns the tensor.

C. Tensor Cache Mechanisms and Optimization

1) *Deduplicating Tensors and Excluding Weights*: Tensor cache has a `get_id()` function to assign a unique identifier to each tensor. The shortcoming of PyTorch native `id()` is that its returned value is related to the GPU memory address. As SSDTrain offloads activations, the latter will be cleared by garbage collection once the control flow goes out of its use scope. The GPU memory address may be reused, causing identifier collision. To solve this, `get_id()` combines the timestamp when it first processes the tensor with the tensor shape as the unique identifier: When `get_id()` processes a tensor t for the first time, `get_id()` adds the current timestamp as an additional attribute to the tensor’s underlying storage `t.untyped_storage()` instead of `t`. This is because sometimes PyTorch creates new `torch.Tensor` objects representing the identical tensor. All future `get_id()` calls get the attribute value. This deduplicating scheme helps prevent redundant I/Os.

PyTorch registers all needed tensors in backward propagation into the computational graph, involving activations and weights. As this work focuses on activations, the tensor cache excludes the weights. To this end, before training, the tensor cache records the identifiers of all weights. As linear layers store the transpose of weights for backward propagation, the unique identifiers of the transpose are recorded. One benefit of our `get_id()` scheme is that the identifier for the transpose of the same weights tensor remains consistent across steps. This is because the transpose uses the original tensor’s underlying storage, which we already assigned a timestamp to before training.

2) *Offloading and Forwarding Tensors*: The tensor cache has two thread pools—one for storing tensors and the other for loading tensors. Submitted jobs are executed in first-in-first-out (FIFO) order.

To hide the I/O latency, the tensor cache starts prefetching each activation before the corresponding module’s backward propagation. The activations in the last module are kept in GPU memory so they need not be prefetched. This simple scheme suffices because in PyTorch, CPU submits GPU kernel launches and memory operations ahead of GPU execution. Prefetching schemes are equivalent as long as there are always I/O tasks in GPU job queue to keep PCIe busy.

Upon loading a tensor, if it is still being stored, the tensor cache will return its in-memory reference to skip loading from SSD. We call this data forwarding. E.g., in Fig. 4, when PyTorch retrieves x from the `Mu1BWD` node, if x is still being stored, it is in memory. Instead of loading the tensor, the tensor cache returns x ’s in-memory

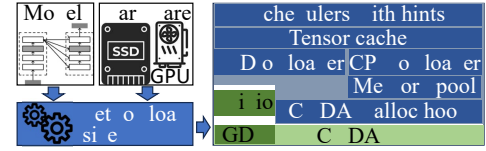


Fig. 3. SSDTrain workflow. SSDTrain components are shown as blue blocks.

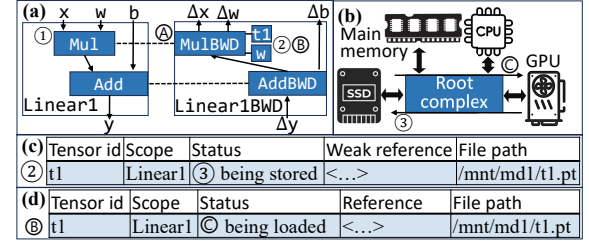


Fig. 4. Tensor cache registers hooks to offload tensors and reload tensors. (a) shows the computational graph. (b) shows the hardware data path. (c) and (d) show the tensor cache state when the pack or unpack hook is triggered.

Algorithm 1: Pack–unpack hook pair used by the tensor cache.

Input: Tensor cache `tc`, current scope module, tensor to pack t , and/or object to unpack `obj`.

```

1 Function pack_hook( $t$ ):
2   if tc.is_weights( $t$ ) or t.is_cpu or
   math.prod( $t.size()$ ) < 2*20 then return  $t$ 
3   tid = get_id( $t$ )
4   tc.add_to_current_scope(tid)
5   if tc.is_offload_amount_reached() or
   tc.is_current_in_backward() then
6     tc.keep_in_gpu_memory(tid,  $t$ )
7   else tc.offload(tid,  $t$ )
8   return tid
9 Function unpack_hook(obj):
10  if isinstance( $obj$ , torch.Tensor) then return obj
11  tc.load_or_wait_load( $obj$ )
12  return tc.get_loaded_tensor( $obj$ )

```

reference converted from the weak reference and store the obtained reference in the tensor cache for future if it is used in other scopes.

D. SSD Write Amount, Bandwidth, and Lifespan

To confirm if our design is viable in large-scale systems, particularly concerning SSD endurance and required bandwidth, we conduct performance modeling to obtain forward propagation time per training step and the size of activations produced in the process. We extend the performance model package `llm-analysis` [54], which models the forward propagation of each transformer layer as a simple pipeline: $t = \max(\sum_l \max(t_{l,compute}, t_{l,memory}), t_{ZeRO,communicate})$, where l denotes any layers inside a transformer layer. When ZeRO is enabled, the ZeRO communication time is assumed to be perfectly pipelined with the non-ZeRO operations at the transformer layer level.

We model the required PCIe write bandwidth per GPU as the total amount of activations divided by half the training time. The lifespan is then projected as $t_{life} = S_{endurance} \cdot t_{step} / S_{activations}$ where $S_{endurance}$ is the lifetime writes allowed by the SSD endurance rating, $S_{activations}$ is the amount of activations per training step, and t_{step} is the step time. We validated the $S_{activations}$ formula with experiments in Sec. IV. We assume four Samsung 980 PRO 1TB for each GPU, and assume the WAF is 2.5 in JESD rating and 1 in our scenario. We also relax the data retention period: NAND flash gets $86 \times$ PE cycles when the data retention period is relaxed from 3 years

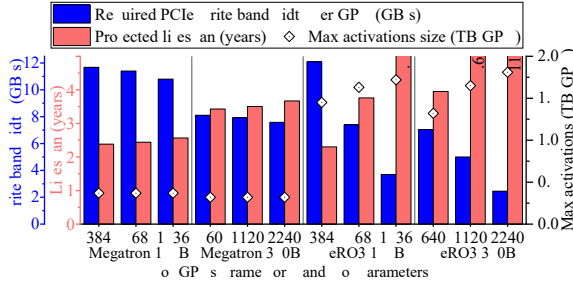


Fig. 5. Estimate of SSD lifespan, PCIe write bandwidth and maximal activations size per GPU. Lifespans longer than 5 years are shown on top of the pink bars. ZeRO3 stands for DeepSpeed with stage-3 ZeRO.

TABLE II
EVALUATION SYSTEM CONFIGURATION.

CPU (Memory)	2× AMD EPYC 7702 64-core (DDR4-3200 1TB)
GPU	2× Nvidia A100 40GB PCIe with NVLink
SSD	7× Intel Optane P5800X 1.6TB. 2× RAID0 arrays.
Software	Ubuntu 20.04.6 (5.15.0-113), CUDA 12.2 (driver 535.183.01), PyTorch 2.2.2, DeepSpeed 0.14.2, Megatron-DeepSpeed [28] (latest), kvikio 24.08

to 1 day [55]–[58]. With these, we obtain Fig. 5. We use measured data from Megatron-LM [10]. GPUs are A100 PCIe. Among all cases, the projected lifespan is more than 2 years, and the write bandwidth per GPU is no greater than 12.1GB/s. Moreover, when the system size and/or the model size scales up, the required PCIe write bandwidth reduces, and the projected lifespan increases. This is because larger systems imply increased communication overhead and reduced computation efficiency, slowing down training on GPUs.

We also estimate the maximal activations size each GPU produces per step by assuming only two layers in a row are in GPU memory at the same time while all other activations are offloaded. Then, diamond marks show the activation a largest micro-batch produces in a step in Fig. 5. The maximal activations size per GPU ranges from 0.4 TB to 1.8 TB, while the micro-batch size ranges from 8 to 32. Activations so large can no longer be held by the main memory and therefore SSD is the only choice as offloading target.

IV. EVALUATION

A. Experimental Setup

We use a machine with 2× A100 PCIe GPUs and 7× Intel P5800X SSDs, as Table II specifies. The SSDs are organized into two RAID0 arrays: one with 3 SSDs, and the other with 4 SSDs. Each array is the dedicated target of one A100. We measured the memory use of the A100 with 4 SSDs during evaluation. For consistency, the GPUs are locked at base frequency. The latest Megatron-DeepSpeed [28] is used, which incorporates DeepSpeed techniques into Megatron.

We measure the pretraining performance on BERT [31] as an encoder-only model, GPT [29] as a decoder-only model, and T5 [32] as an encoder-decoder model. We use the OSCAR dataset [59], [60].

We use the two GPUs for TP. The number of micro-batches per step is fixed at 1 because without PP, a new micro-batch will not start before both forward propagation and backward propagation of the previous micro-batch are done. More micro-batches only brings in gradient accumulation and does not affect the activation offloading pattern. In other words, unless stated otherwise, the micro-batch size is equivalent to batch size throughout Sec. IV. The hidden dimension is from 8192 to 16384, and we use typical hyperparameters [31], [32], [61] for this range. The attention head dimension is 128. The

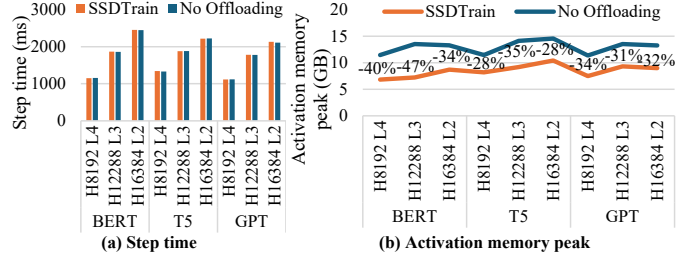


Fig. 6. Comparing SSDTrain to runs without tensor offloading. We test with different hidden dimensions (H) and number of layers (L). Batch size is 16.

text sequence length is 1024. For T5, the number of decoders is half of the total number of layers, rounded down. FlashAttention-2 [62] is used with or without SSDTrain for optimized attention computation.

As each A100 has only 40GB of device memory, to explore the design space closer to that in real-world training systems with A100 80GB and later GPUs [7], [10], we make several mitigations. First, we use FP16 instead of mixed precision, eliminating the FP32 weight copy. Second, we use SGD instead of Adam as the optimizer to reduce optimizer states. The two measures only affect accumulation operations and weight updates, thus imposing a constant bias in step time and memory use in execution with or without SSDTrain.

B. Performance and Peak Memory Usage

To understand SSDTrain’s impact on execution time and peak memory usage, we measure the step time of BERT, T5, and GPT and the memory peak during forward and backward propagation. The collected metrics of system with SSDTrain and without are compared in Fig. 6. For each model, we collected three scenarios with different (hidden dimension, number of layers): (8192, 4), (12288, 3) and (16384, 2). As shown, SSDTrain has almost no performance overhead in all cases. Although SSDTrain and its optimizations introduce additional CPU logic, the performance comparison indicates that this logic is not on the critical path. Rather, GPU computation defines the critical path, and the CPU’s role lies primarily in launching new GPU jobs before current GPU operations complete. Thus, the CPU is underutilized, and SSDTrain’s extra work does not lead to delay in new tasks reaching the GPUs. SSDTrain effectively reduces the activations’ memory use peak by 28%–40% in these cases.

C. Comparing the Activations Placement Strategies

SSDTrain opens up offloading activations to SSDs as an option besides keeping activations in the GPU memory, and activations checkpointing. We compare the three different strategies here on the recompute-offload-keep (ROK) curve: Fig. 7 shows the training of two 3-layer BERT models. One set the hidden dimension as 12K and the other set it as 14K. In a ROK curve, each run is represented by a point. The x-axis is the activations memory peak. The y-axis is the model throughput [10], i.e., the number of algorithmic computations involved in the training step regardless of software and hardware implementation, e.g., whether the activations are recomputed, divided by the training step time. In these two cases, SSDTrain reduces the GPU activations memory peak, allowing for a larger batch size to attain higher throughput. Given the same batch size, SSDTrain offloading attains the throughput the same as the throughput when the activations are kept in memory. Meanwhile, SSDTrain gets a lower activations memory peak than the recomputation. Compared with keeping the activations in memory, SSDTrain is able to double the batch size with the same activations memory budget.

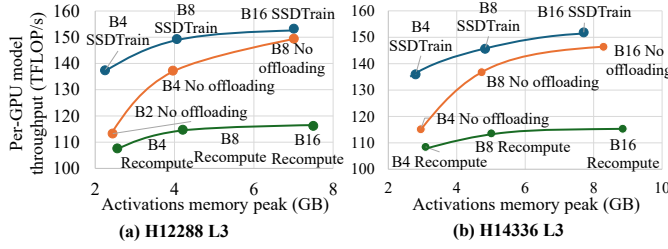


Fig. 7. Recompute-offload-keep (ROK) curve of 3-layer (L) BERT with hidden dimension (H) as 12K or 14K. Different batch sizes (B) are tested.

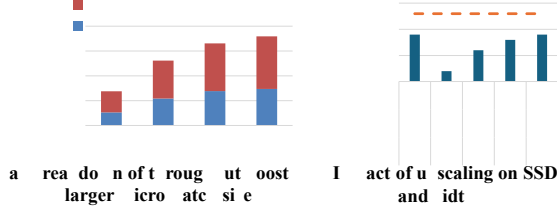


Fig. 8. Case study of 3-layer BERT with hidden dimension as 12K.

TABLE III

THE PER-GPU OFFLOADED TENSOR AMOUNT, ESTIMATE, AND REQUIRED PCIe WRITE BANDWIDTH OF BERT WITH DIFFERENT HIDDEN DIMENSIONS (H) AND NUMBER OF LAYERS (L). BATCH SIZE IS 16.

	H8192 L4	H12288 L3	H16384 L2
Offloaded amount	10.37 GB	12.85 GB	10.75 GB
Model estimate	11.13 GB	12.60 GB	11.50 GB
PCIe write bandwidth	18.0 GB/s	13.8 GB/s	8.76 GB/s

Other than the three strategies, before FlashAttention [63], Megatron [8] proposed selective checkpointing that recomputes the core attention modules. As we use FlashAttention, the core attention module is done in one kernel, eliminating these intermediate tensors. The effect of selective checkpointing with FlashAttention has negligible impact on performance and peak memory usage for activations.

D. Discussion

Examining the modeling. To understand the accuracy of the model in Sec. III-D, we compare SSDTrain’s offloaded amount with the model estimate. As Table III shows, the figures are close. We also compute the required PCIe write bandwidth, which is reduced when the hidden dimension gets larger. Typically, a model with more than 60B parameters has a hidden dimension of no less than 8K [35], [61]. PCIe write bandwidth aligns with the estimate in Sec. III-D.

Impact of larger micro-batch size. To further understand how larger micro-batch size improves the performance, we compare the no-offloading cases in Figure 7(a) to the same configurations with batch size as 1 and break down the throughput improvement in Fig. 8(a). The improvement primarily comes from time saving by weights update, which is very relevant to large-scale LLM training systems. The micro-batch size is usually set small, e.g., 1 or 2 in Paxml [64] and BLOOM [11] pretraining, in exchange for smaller bubbles introduced by PP. In the BLOOM training system, each data parallel rank is assigned a mini-batch with 32 samples. When the micro-batch size is no less than 4, the ideal PP bubble time percentage is no less than 11.5%. However, weight update and gradient accumulation cost is inversely proportional to the micro-batch size, which is huge when the micro-batch size is 1 or 2. SSDTrain allows larger micro-batch sizes given the same activation memory budget, thus beneficial to these PP-enabled training systems.

Impact of upscaling. Sec. II-B demonstrates that the whole-system activations size $S_{activations}$ grows slower than the whole-system GPU throughput C , i.e., $S_{activations} \propto C^{\frac{5}{6}}$. Therefore, the bandwidth required to fully overlap the computation with the SSD accesses is reduced. In short, the scaling of LLM is essentially a weak scaling scenario, and the SSD IO latency is easier to hide when it is scaled up. In Fig. 8(b), we further project the impact of upscaling on the write bandwidth per GPU using llm-analysis. We follow typical parallelism configurations [10], [64] when the number of GPUs is less than 100. In all projected cases, the write bandwidth per GPU is smaller than the original 2-GPU case (orange dashed line). Vanilla DP only affects weights update and therefore has no effect on the write bandwidth. ZeRO may reduce the write bandwidth requirement due to the communication incurred in forward and backward propagation. **Cost analysis.** We study the SSD cost associated with adopting SSDTrain offloading. To get the endurance in Fig. 5, each A100 priced at US\$10K [65] is paired with in total US\$360 worth of SSDs. The evaluation uses 7 Intel P5800X for the 2 A100s. Although P5800X is more expensive, the price per PBW is comparable [66].

V. RELATED WORK

Many LLM systems with offloading abilities are inference-only [46], [47], [67]. In inference, weights and KV-cache never change and are reused across iterations; this is leveraged to enhance locality and memory efficiency. However, in training, all tensors, including the weights, change across the iterations. Some work avails offloading [48] for training but is mostly designed to fit in larger models at the cost of performance. Async transfer is missing to keep performance. Another direction is to offload computation to the CPU [22]–[24]. The offloaded computation is light, and the offloaded data include gradients, sparse elements in the weights, etc. Our work is orthogonal: Interference with the CPU is minimized because we offload the activations to SSDs via GDS. Activations are for compute-intensive gradient computation, which is best done solely on GPUs.

Before LLMs, there is work on offloading for deep learning [25], [68]–[71]: Most target main memory while some [25] target SSDs. LLM is unique because massive parallelism and its memory implications are fundamental to the design space. SSDTrain naturally supports multiple GPUs. We showed its viability on clusters. Besides, LLM’s demand for computing power is so high that it stimulates rapid development in specialized hardware and frameworks. SSDTrain ensures good interoperability, while most prior work is bound to a specific PyTorch version or a custom runtime supporting select layers.

VI. CONCLUSION

In LLM training, activations dominate the increasingly limited GPU memory. To address this, we propose SSDTrain as an adaptive activation offloading framework to SSDs. We demonstrate its viability in large-scale systems by modeling. The evaluation shows SSDTrain reduces the activations peak memory use by up to 47% with negligible overhead. We analyze how this may lead to increased throughput by increasing micro-batch size and reducing pipeline bubbles.

ACKNOWLEDGEMENTS

This work was partially supported by the IBM-ILLINOIS Center for Cognitive Computing Systems Research (C3SR), by the IBM-ILLINOIS Discovery Accelerator Institute (IIDAI), and by NVIDIA. This work was also partly supported by credits from Google Cloud, the Illinois Computes program, and the National Energy Research Scientific Computing Center (NERSC ERCAP0031237).

REFERENCES

- [1] OpenAI. (2022) ChatGPT. [Online]. Available: <https://chatgpt.com/>
- [2] Microsoft. (2023) Bing Chat — Microsoft Edge. [Online]. Available: <https://www.microsoft.com/en-us/edge/features/bing-chat>
- [3] Midjourney. (2022) Midjourney. [Online]. Available: <https://www.midjourney.com/website>
- [4] LangChain. (2022) LangChain. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [5] J. Wei *et al.* (2022) Emergent Abilities of Large Language Models. [Online]. Available: <http://arxiv.org/abs/2206.07682>
- [6] D. Meyer. (2024) The cost of training AI could soon become too much to bear. Yahoo Finance. [Online]. Available: <https://finance.yahoo.com/news/cost-training-ai-could-soon-101348308.html>
- [7] Z. Liu *et al.*, “Winner-Take-All Column Row Sampling for Memory Efficient Adaptation of Language Model,” Dec. 2023.
- [8] V. Korthikanti *et al.*, “Reducing Activation Recomputation in Large Transformer Models,” May 2022.
- [9] Z. Jiang *et al.*, “MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs,” Feb. 2024.
- [10] M. Shoeny *et al.*, “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” Mar. 2020.
- [11] T. L. Scao *et al.*, “BLOOM: A 176B-Parameter Open-Access Multilingual Language Model,” Jun. 2023.
- [12] L. Chen, “Dissecting Batching Effects in GPT Inference,” <https://le.qun.ch/en/blog/2023/05/13/transformer-batching/>, 2023.
- [13] Q. Anthony *et al.*, “The Case for Co-Designing Model Architectures with Hardware,” Jan. 2024.
- [14] R. Y. Aminabadi *et al.*, “DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale,” Jun. 2022.
- [15] J. Kaplan *et al.*, “Scaling Laws for Neural Language Models,” Jan. 2020.
- [16] S. McCandlish *et al.*, “An Empirical Model of Large-Batch Training,” Dec. 2018.
- [17] T. Chen *et al.*, “Training Deep Nets with Sublinear Memory Cost,” Apr. 2016.
- [18] The Epoch AI, “Announcing Epoch AI’s Updated Parameter, Compute and Data Trends Database,” Oct. 2023.
- [19] Microsoft, “ND A100 v4-series - Azure Virtual Machines,” Feb. 2024.
- [20] Google, “GPU machine types | Compute Engine Documentation,” 2017.
- [21] NCSA, “Delta Project Profile,” <https://www.ncsa.illinois.edu/research/project-highlights/delta/>, 2022, accessed 07/21/2024.
- [22] K. Kamahori *et al.* (2024) Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models.
- [23] J. Ren *et al.*, “ZeRO-Offload: Democratizing Billion-Scale Model Training,” in *USENIX ATC*, 2021.
- [24] Y. Song *et al.* (2023) PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU. [Online]. Available: <http://arxiv.org/abs/2312.12456>
- [25] J. Bae *et al.*, “FlashNeuron: SSD-Enabled Large-Batch Training of Very Deep Neural Networks,” in *FAST*, 2021.
- [26] Google, “About google cloud hyperdisk — compute engine documentation,” <https://cloud.google.com/compute/docs/disks/hyperdisks>, 2022.
- [27] G. K. Lockwood *et al.*, “Architecture and performance of Perlmutter’s 35 PB Clusterstor E1000 all-flash file system,” *CCPE*, p. e8143, 2024.
- [28] Microsoft. (2019) Megatron-DeepSpeed: Ongoing research training transformer language models at scale, including: BERT & GPT-2.
- [29] A. Radford *et al.* (2019) Language Models are Unsupervised Multitask Learners.
- [30] A. Vaswani *et al.*, “Attention is All you Need,” in *NeurIPS*, vol. 30. Curran Associates, Inc., 2017.
- [31] J. Devlin *et al.*, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” May 2019.
- [32] C. Raffel *et al.*, “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” Sep. 2023.
- [33] Y. Xu *et al.*, “GSPMD: General and Scalable Parallelization for ML Computation Graphs,” Dec. 2021.
- [34] S. Rajbhandari *et al.*, “ZeRO: Memory optimizations Toward Training Trillion Parameter Models,” in *SC20*. IEEE, Nov. 2020, pp. 1–16.
- [35] Jordan Hoffmann *et al.* (2022) Training Compute-Optimal Large Language Models. [Online]. Available: <http://arxiv.org/abs/2203.15556>
- [36] T. B. Brown *et al.*, “Language models are few-shot learners,” 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [37] Samsung, “Ultra-Low Latency with Samsung Z-NAND SSD,” <https://download.semiconductor.samsung.com/resources/brochure/Ultra-LowLatencywithSamsungZ-NANDSSD.pdf>, 2017.
- [38] JEDEC, *JESD218B: Solid-State Drive (SSD) Requirements and Endurance Test Method*, Std., 2016.
- [39] Lenovo. (2023) What do I need to know about SSD endurance and overprovisioning? [Online]. Available: <https://thinksystem.lenovofiles.com/storage/help/index.jsp?topic=%2Fde-series-olh-11.80%2Fwhat-do-i-need-to-know-about-ssd-endurance-and-overprovisioning.html>
- [40] QNAP Systems, Inc., “QNAP NAS Solution: QTS SSD Extra Over-Provisioning,” 2018.
- [41] SMART Modular Technologies, Inc. (2024) Why SMART’s Over-Provisioning? [Online]. Available: <https://www.smartm.com/technology/over-provisioning>
- [42] Solidigm. (2022) Solidigm™ SSD Endurance Estimator. [Online]. Available: <https://estimator.solidigm.com/ssdendurance/index.htm>
- [43] Intel. (2018) Over-Provisioning NAND-Based Intel® SSDs for Better Endurance. [Online]. Available: <https://www.intel.com/content/www/us/en/docs/over-provisioning-nand-based-ssds-better-endurance-whitepaper.pdf>
- [44] Samsung, “Over-Provisioning Benefits for Samsung Data Center SSDs,” 2019. [Online]. Available: <https://download.semiconductor.samsung.com/resources/white-paper/S190311-SAMSUNG-Memory-Over-Provisioning-White-paper.pdf>
- [45] S. Maneas *et al.*, “Operational Characteristics of SSDs in Enterprise Storage Systems: A Large-Scale Field Study,” in *FAST*, 2022.
- [46] Y. Sheng *et al.*, “FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU,” Jun. 2023.
- [47] K. Alizadeh *et al.*, “LLM in a flash: Efficient Large Language Model Inference with Limited Memory,” Jan. 2024.
- [48] S. Rajbhandari *et al.*, “ZeRO-infinity: Breaking the GPU memory wall for extreme scale deep learning,” in *SC*, Nov. 2021, pp. 1–14.
- [49] D. Inupakutika *et al.*, “Quantifying Performance Gains of GPUDirect Storage,” in *NAS*, 2022, pp. 1–9.
- [50] H. Yang *et al.* (2024) ProTrain: Efficient LLM Training via Memory-Aware Techniques. [Online]. Available: <http://arxiv.org/abs/2406.08334>
- [51] X. Sun *et al.*, “STRONGHOLD: Fast and affordable billion-scale deep learning model training,” in *SC*, 2022, pp. 1–17.
- [52] Nvidia, “RapidSai/kvikio: KvikIO - High Performance File IO,” <https://github.com/rapidsai/kvikio>, 2022, accessed 07/21/2024.
- [53] Wikipedia, “Monkey patch,” 2006.
- [54] C. Li, “LLM-Analysis: Latency and Memory Analysis of Transformer Models for Training and Inference,” 2023.
- [55] Y. Cai *et al.*, “Flash correct-and-refresh: Retention-aware error management for increased flash memory lifetime,” in *ICCD*, 2012.
- [56] —, “Error patterns in mlc nand flash memory: Measurement, characterization, and analysis,” in *DATE*. IEEE, 2012, pp. 521–526.
- [57] R.-S. Liu *et al.*, “Optimizing NAND Flash-Based SSDs via Retention Relaxation,” in *FAST*. USENIX Association, 2012.
- [58] S. Kim *et al.*, “Behemoth: A Flash-centric Training Accelerator for Extreme-scale dnns,” 2021, pp. 371–385.
- [59] P. J. O. Su’arez *et al.*, “A monolingual approach to contextualized word embeddings for mid-resource languages,” in *ACL*, Jul. 2020.
- [60] —, “Asynchronous pipelines for processing huge corpora on medium to low resource infrastructures,” ser. CMLC-7. Mannheim: Leibniz-Institut für Deutsche Sprache, 2019, pp. 9 – 16.
- [61] H. Touvron *et al.*, “Llama 2: Open Foundation and Fine-Tuned Chat Models,” Jul. 2023.
- [62] T. Dao. (2023) FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.
- [63] T. Dao *et al.* (2022) FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.
- [64] Nitin *et al.* (2023) Scaling large language model training with Pax on GPUs.
- [65] Dihuni. (2021) NVIDIA A100 40GB Ampere PCIe GPU.
- [66] Newegg. (2021) Intel optane p5800x 1.6TB.
- [67] W. Kwon *et al.*, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *SOSP*. ACM, 2023.
- [68] X. Peng *et al.*, “Capuchin: Tensor-based GPU Memory Management for Deep Learning,” in *ASPLOS*. ACM, Mar. 2020, pp. 891–905.
- [69] L. Wang *et al.*, “SuperNeurons: Dynamic GPU Memory Management for Training Deep Neural Networks,” in *PPOPP*, Feb. 2018, pp. 41–53.
- [70] M. Rhu *et al.*, “vDNN: Virtualized Deep Neural Networks for Scalable, Memory-Efficient Neural Network Design,” Jul. 2016.
- [71] C.-C. Huang *et al.*, “SwapAdvisor: Pushing Deep Learning Beyond the GPU Memory Limit via Smart Swapping,” in *ASPLOS*, Mar. 2020.